

パズルゲームでアルゴリズム の使われ方を見てみよう

2025/09/07 KC3 2025

RiPPro MMRZ (紺野 拓磨)

目次

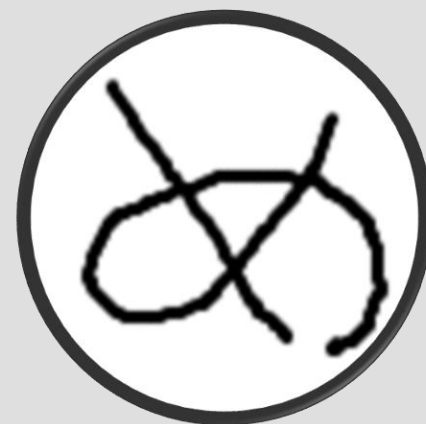
1. はじめに
 1. 自己紹介
 2. サークル紹介
 3. 本プログラムについて
 1. Siv3D の紹介
2. 探索アルゴリズムに触れよう
 1. アルゴリズムとは
 2. パズルゲーム例：マインスイーパー
3. 実際にパズルゲームの作り方を考えてみる
 1. ゲームのルール
 2. 盤面の作り方

1. はじめに

紹介を行うセクションです

1-1. 自己紹介

- 名前 : mmrz (紺野 拓磨)
- 所属 : 立命館大学 情報理工学部 知能情報コース 4 年
 - 卒業研究として推薦システムを取り扱っています 📖
- 所属サークル : RiPPro (+ RiST, RiG++)
- SNS
 - X : @MM_RZ_AC
 - Discord : mm_rz
- 趣味 : 最近は Board Game Arena を漁っています



1-2. サークルの自己紹介

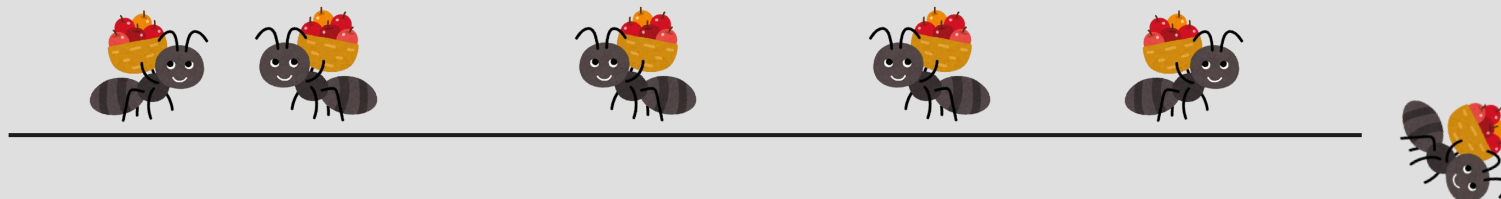
- 正式名称 : 情報理工学部 プロジェクト団体 RiPPro
- 活動内容 : 定期的な競技プログラミングコンテストへの参加・交流
- 団員 : 20 人程度
- サークルとしてのKC3参加は久しぶりなようです

競技プログラミングとは

～ “与えられた制約下で**正確さと速さ**を競いながら問題をアルゴリズムと実装で解く”競技～

問題例 “Ants”

長さ L cm の竿の上を n 匹のアリが毎秒 1 cm のスピードで歩いています。アリが竿の端に到達すると竿の下に落ちていきます。また、竿の上は狭くてすれ違えないので、二匹のアリが出会うと、それぞれ反対を向いて戻っていきます。各蟻について、現在の竿の左端からのからの距離 x_i はわかりますが、どちらの方向を向いているのかはわかりません。すべてのアリが竿から落ちるまでにかかる最小の時間と最大の時間をそれぞれ求めなさい。

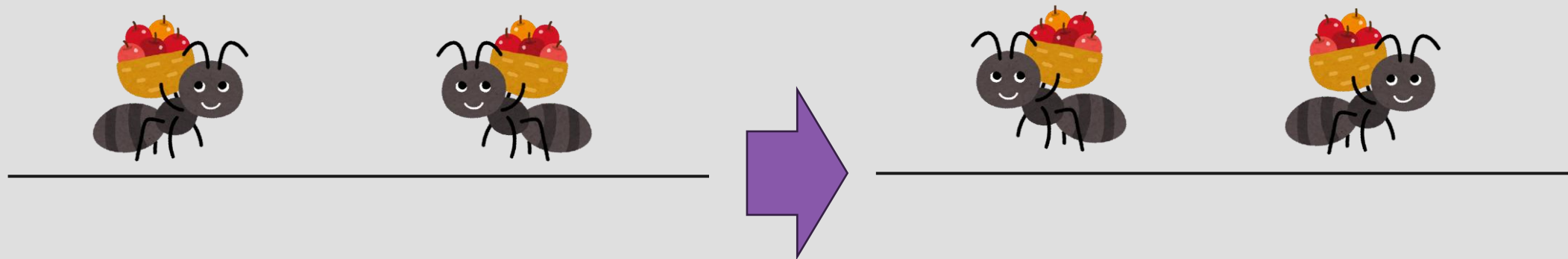


解き方の例(アバウトな計算量の話)

- アリの向きを 2^n パターンすべて試す
- それぞれのパターンにおいてシミュレーション(かかる時間を適当に t とする, $t \geq \frac{L}{2}$)
- 少なくとも $O(2^n t)$ に比例する計算量がかかることが分かった
 - 現代のコンピューターは、 10^8 くらいの処理を 1 秒で行える
 - アリが数匹であれば問題ないが、100匹くらいになると永遠に終わらない (400t 垓年...計算が終わる前に宇宙が終わってしまう)
 - 実は $O(n)$ で計算することができる
 - 少なくともシミュレーションが不要らしい

競プロで問われる“創意工夫”

- アリがすれ違う様子を考える
 - “二匹のアリが出会うと、それぞれ反対を向いて戻っていきます”



アリを区別しないで考えると、**すれ違ったとみなしても状態が変わらない！！** → それぞれ近い端を目指すのが最小, 遠い端を目指すのが最大

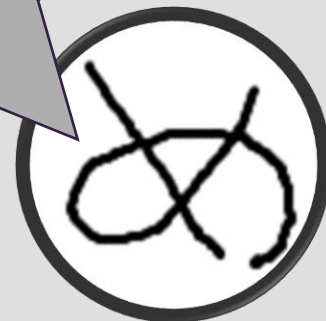
1-3. 本プログラムについて

表題：パズルゲームでアルゴリズムの使われ方を見てみよう

探索アルゴリズムが活躍するシーンを、Siv3Dで制作されたパズルゲームを通して観察します。
(時間に余裕があれば、発展的なアルゴリズムの紹介なども行う予定です！)
ゲームプログラミングやWebアプリ制作の幅が広がることを期待しています。

勉強会概要(<https://kc3.me/study/3878/>)より

生成 AI などによって実装をする機会が減少しているが、そんな中でアルゴリズムを知っている/自力で書けることの意義について伝えられればと思っています！



(補足)サークルと本プログラムの関連性

競プロにどっぷりハマった側ではあるが、開発などに興味がある人を誘うのは気が引けるので、正直やりたくない。

そこで、アルゴリズムが活躍する場面の分かりやすい「パズルゲーム」を題材とした。

開発経験としてもパズルゲームを題材にしてみるのも個人的にはオススメ。

フレームワークとして Siv3D を使うが、事前知識不要。

1-3-1. Siv3D の紹介



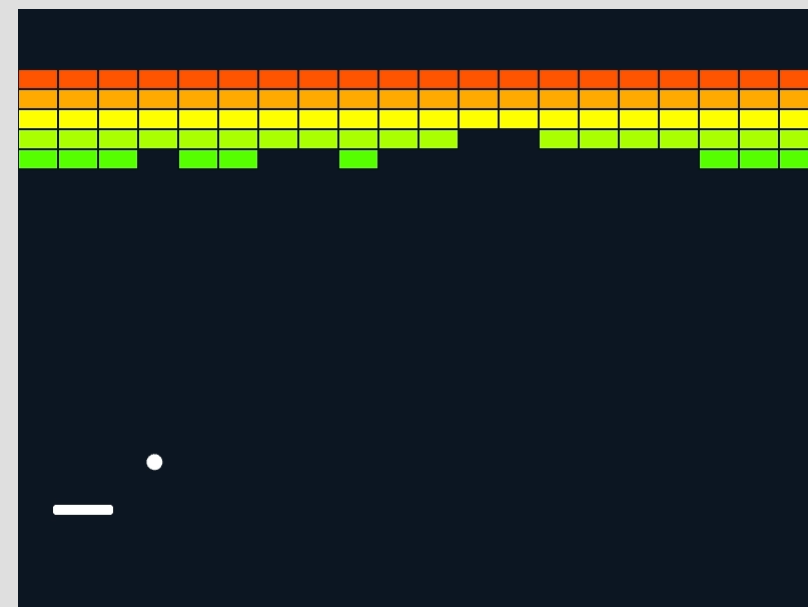
Siv3D (シブスリーディー) は、音や画像、AI を使ったゲームやアプリを、モダンな C++ コードで楽しく簡単に開発できるオープンソースのフレームワークです。豊富なサンプルコードとチュートリアルが用意され、オンラインのユーザコミュニティで気軽に質問や相談ができます。（公式HPより）

魅力的なゲームを、シンプルな実装で実現できるのが特徴。

公式HPの「ゲームのサンプル集」

(<https://siv3d.github.io/ja-jp/samples/games/>)

を眺めるだけでも充分楽しい



公式チュートリアルブロック崩し (35行)

(参考1)Siv3D プロジェクトの作り方

<https://siv3d.github.io/ja-jp/download/windows/#3-siv3d-%E3%83%97%E3%83%AD%E3%82%B8%E3%82%A7%E3%82%AF%E3%83%88%E3%82%92%E4%BD%9C%E6%88%90%E3%81%99%E3%82%8B>

1. OpenSiv3D をダウンロード
2. Visual Studio 2022 を起動
3. 「新しいプロジェクトを作成」
4. 左側の “OpenSiv3D” を選択

(参考2) Siv3D の大まかな実行の流れ

```
# include <Siv3D.hpp>
void Main() {
    // ウィンドウの背景色や、フォントを設定したりする部分

    // メインループ (ウィンドウが閉じられるまで、
    //   モニターのフレームレート分繰り返される)
    while (System::Update()) {
        // 何かの描画や処理をする部分
    }
}
```



サンプルプログラム実行例

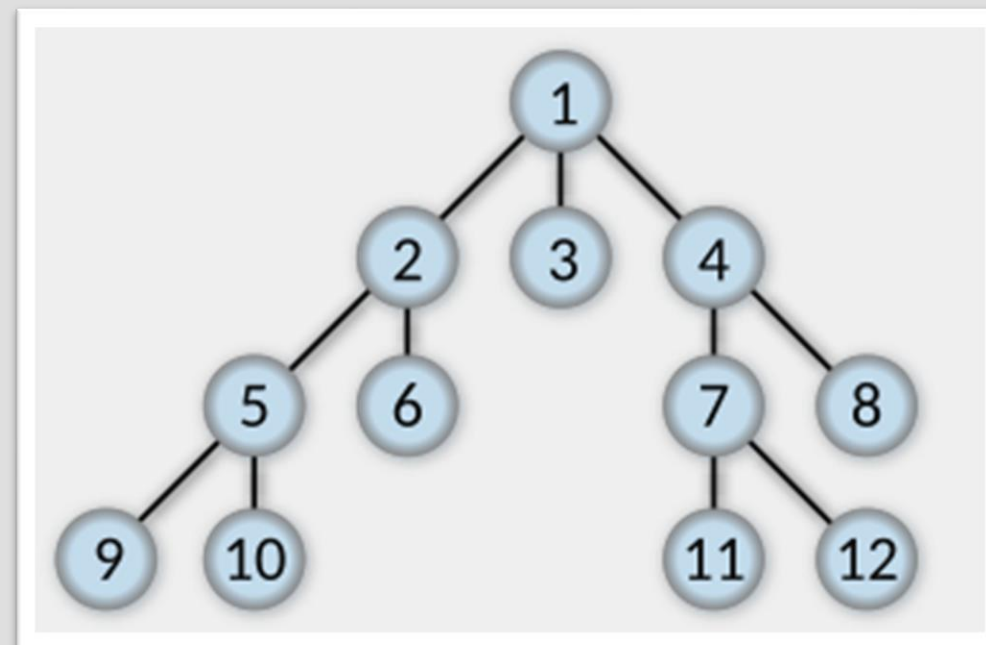
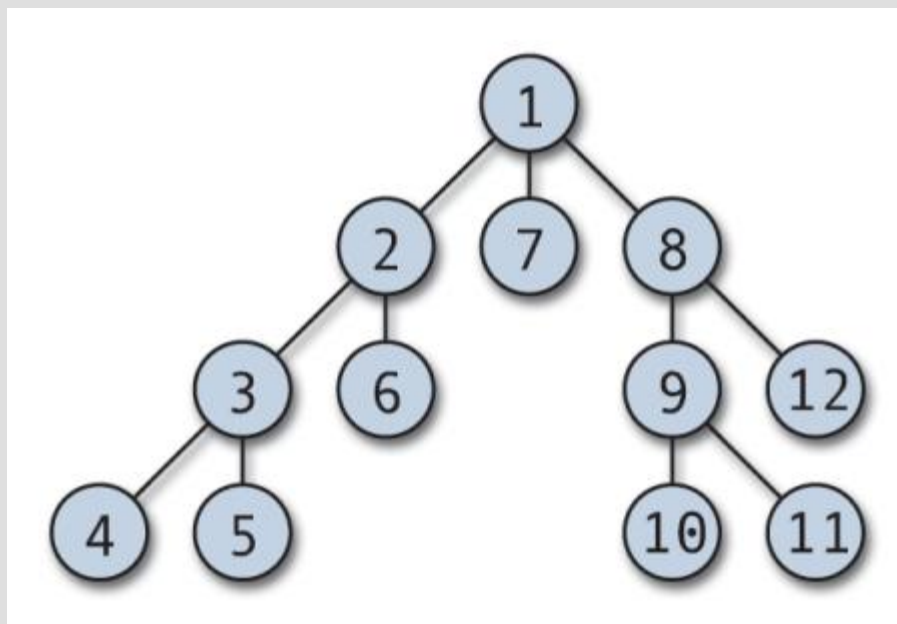
2. 探索アルゴリズムに触れよう

大学の講義でも触れる「探索アルゴリズム」について、実際の活用例を覗いてみましょう

2-1. アルゴリズムとは？

アルゴリズム：“問題を解くための、有限で明確な手順”

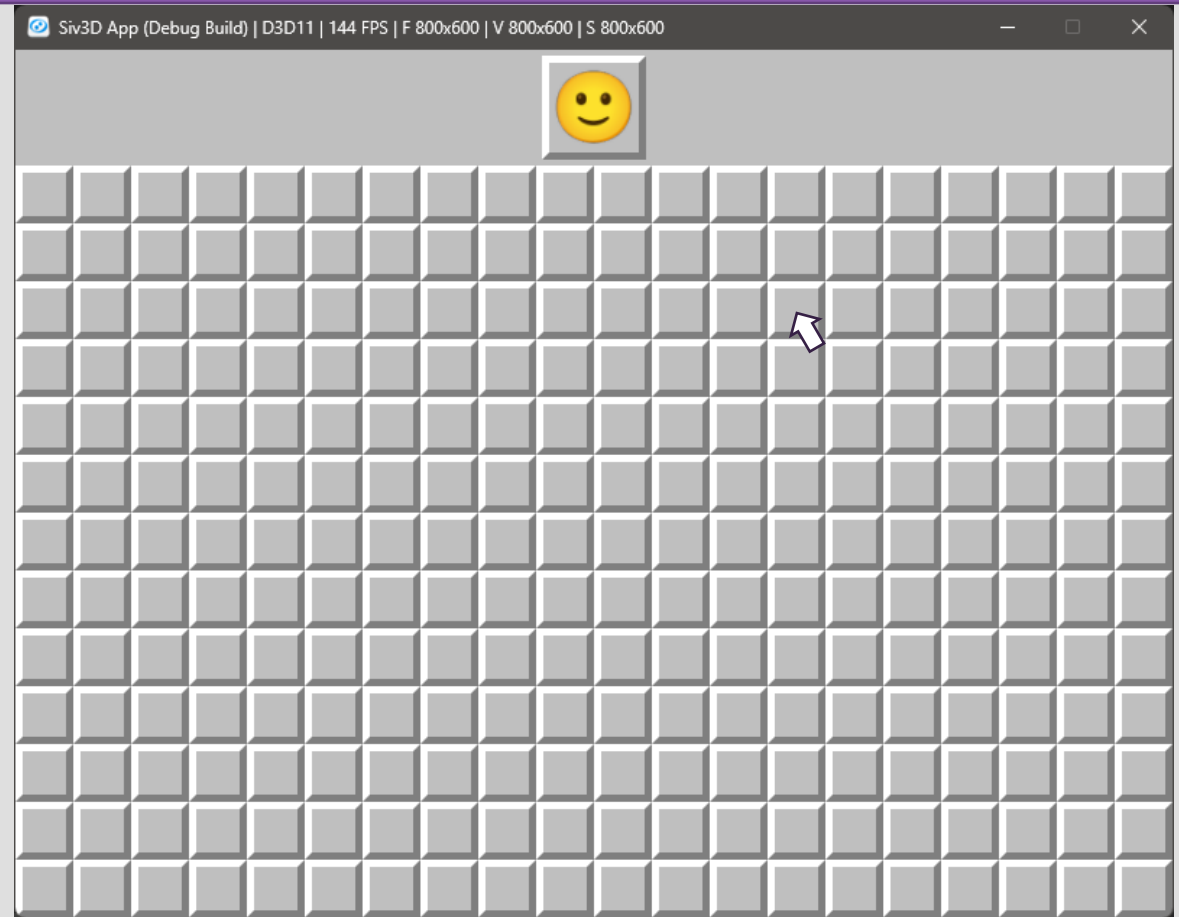
探索アルゴリズム：“多数の候補から目的の要素・状態・経路を見つけるための手順” (例：地図上から最短の経路を求める)



左:深さ優先探索, 右:幅優先探索 教科書にはこういうアバウトな図が載っていがち(図はwikipediaより)

2-2. パズルゲーム例：マインスイーパー

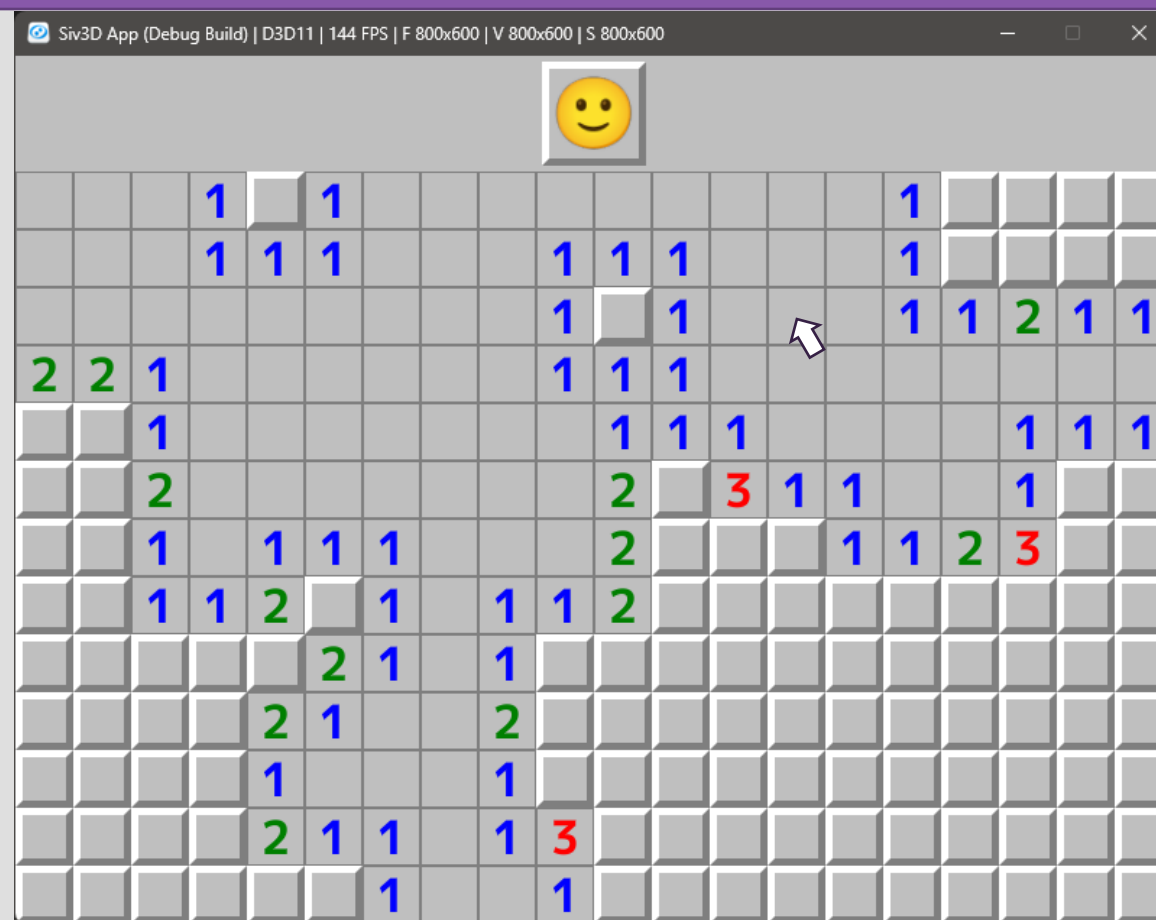
クリックしたら



2-2. パズルゲーム例：マインスイーパー

クリックしたら

展開されるプログラムを作らなければいけない



2-2. パズルゲーム例：マインスイーパー

※注意：後述する「**全探索（深さ優先探索）**」の練習のため、公式実装例を改変した形で説明します

公式実装

~ゲーム開始前~

64行目, MakeGame()で爆弾設置

100行目, MakeStates() で同時に展開されるマス集合を計算

~ゲーム中~

230行目, OpenGroup() で, クリックされたマスが含まれる集合をすべて展開

今回想定する実装

~ゲーム開始前~

64行目, MakeGame()で爆弾設置

100行目, MakeStates() で同時に展開されるマス集合を**全探索**で計算

~ゲーム中~

230行目, OpenGroup() で, クリックされたマスが含まれる集合をすべて展開

展開を深さ優先探索するには

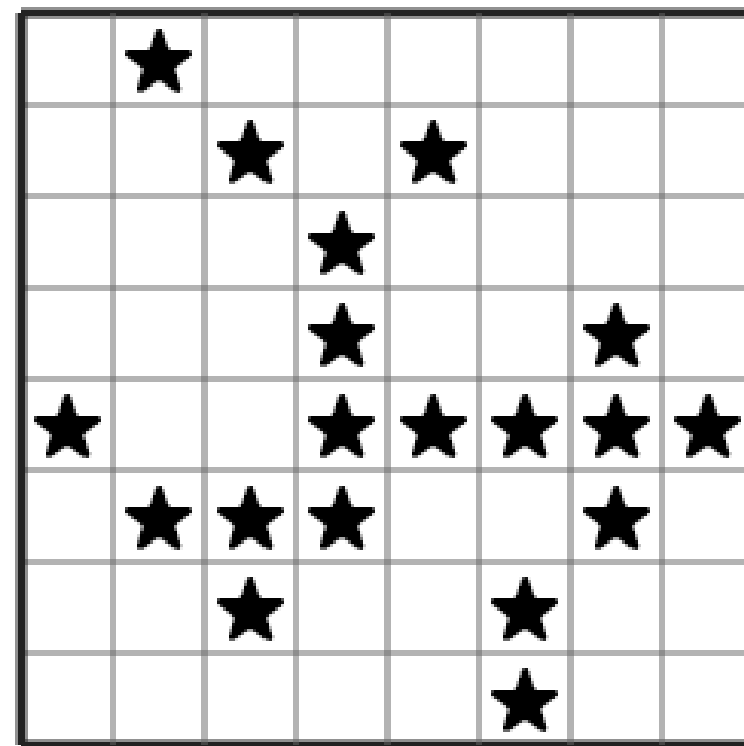
グループを求めるためには右図
のようなことをしたい

0	★	1	1	1	1	1	1
0	0	★	1	★	1	1	1
0	0	0	★	1	1	1	1
0	0	0	★	1	1	★	1
★	0	0	★	★	★	★	★
2	★	★	★	2	2	★	3
2	2	★	2	2	★	3	3
2	2	2	2	2	★	3	3

数字が集合の番号, ★が爆弾を示す

展開を深さ優先探索するには

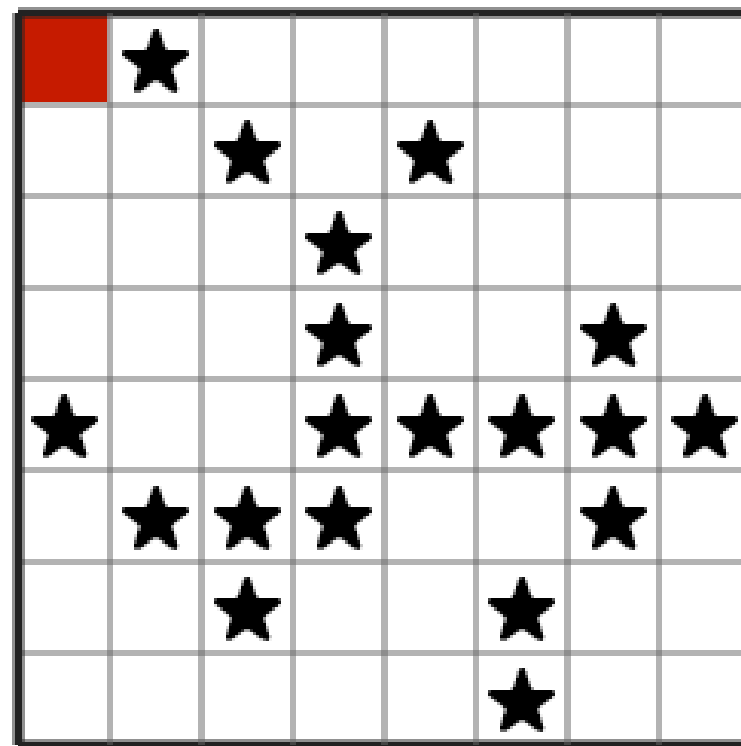
爆弾を置き終わった状態を考える



展開を深さ優先探索するには

左上から順にマス进行査する

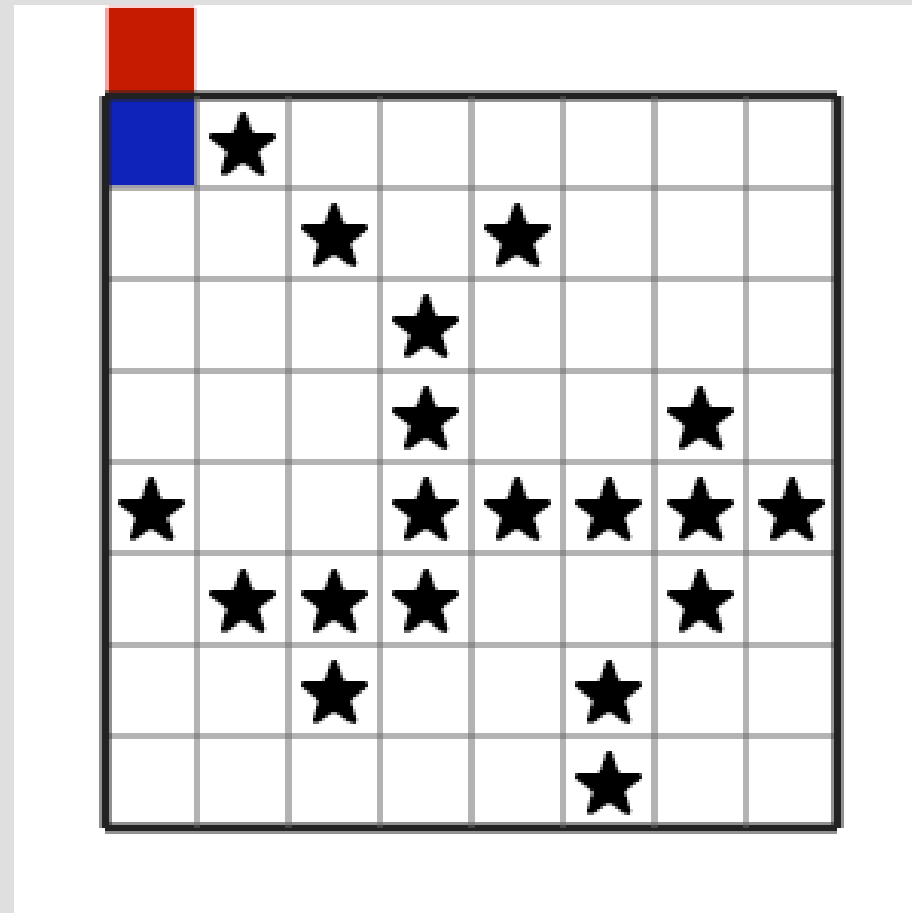
新たに空きマスを発見した(このマスを探索済みにする)



展開を深さ優先探索するには

隣接するマスを確認していく

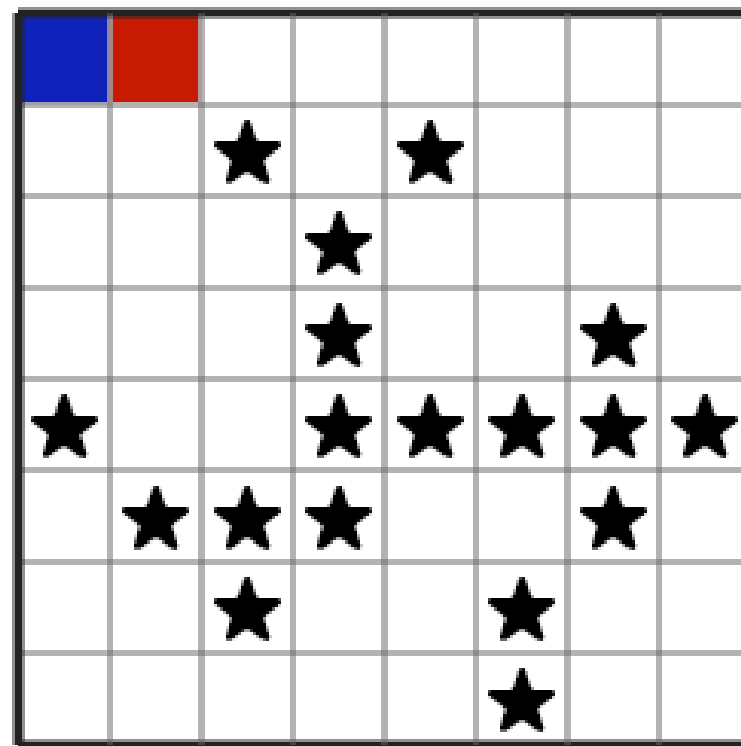
その上は範囲外



展開を深さ優先探索するには

隣接するマスを確認していく

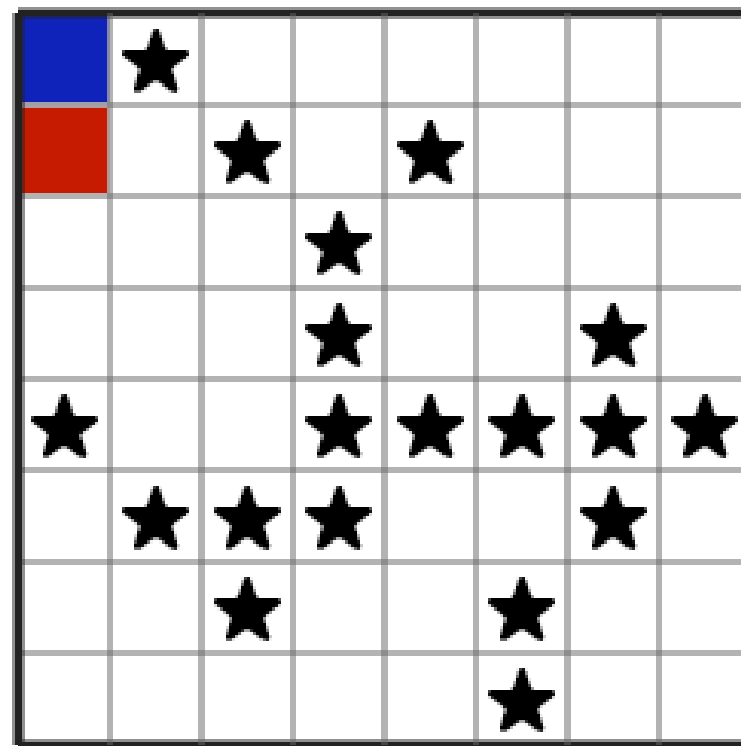
その右は爆弾マス



展開を深さ優先探索するには

隣接するマスを確認していく

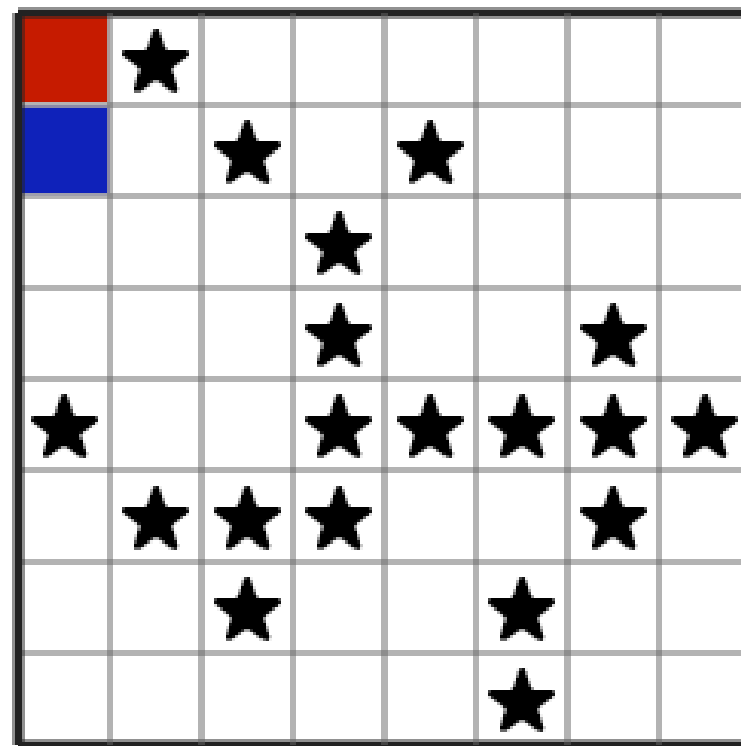
新たに空きマスを発見した(このマスを探索済みにする)



展開を深さ優先探索するには

隣接するマスを確認していく

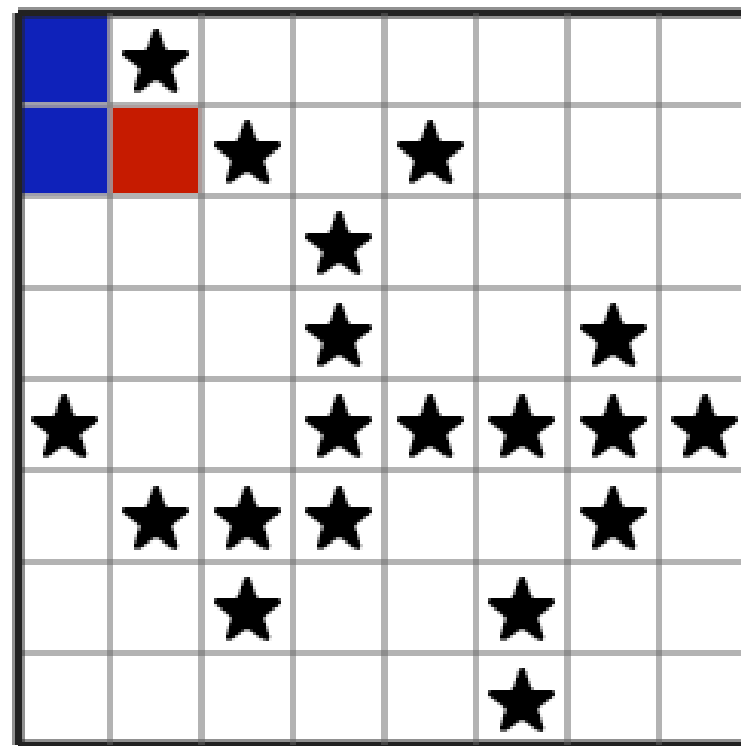
その上は探索済み



展開を深さ優先探索するには

隣接するマスを確認していく

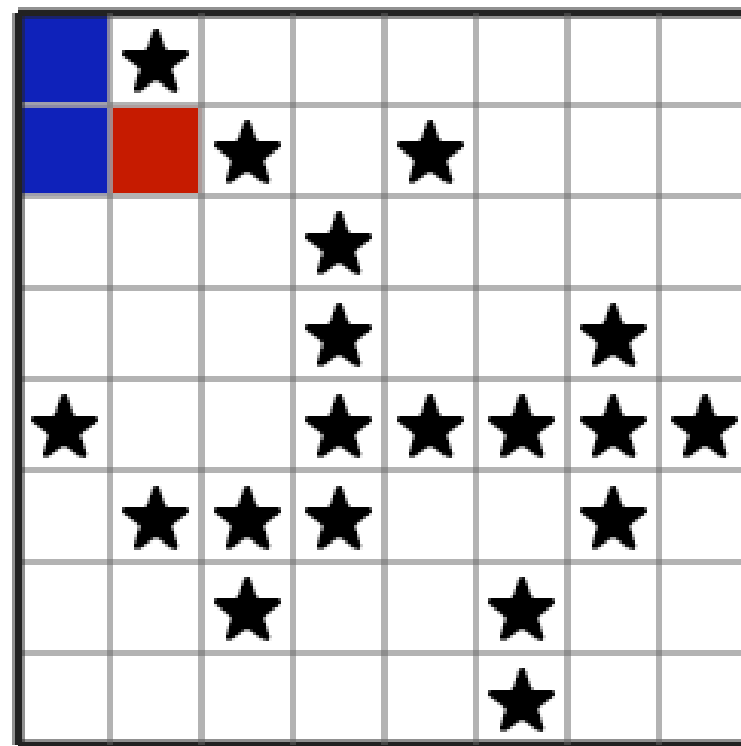
その右は未探索の空きマス



展開を深さ優先探索するには

隣接するマスを確認していく

新たに空きマスを発見した(このマスを探索済みにする)

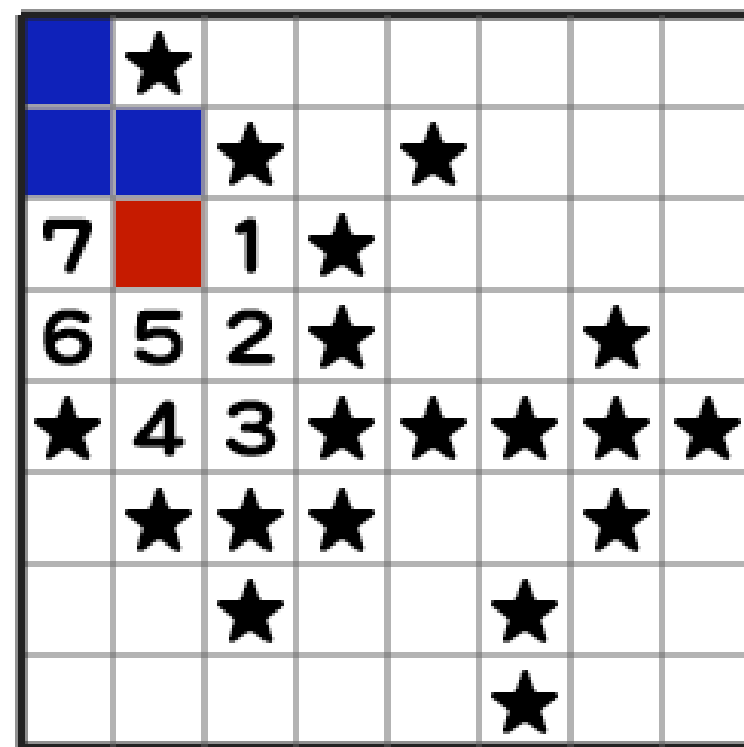


展開を深さ優先探索するには

隣接するマスを確認していく

残りは数字の順に見つかる

...一つの例だけだと伝わりにく
いかもしれないので



プログラムを用意しました

爆弾を自由に設置して、深さ優先探索で同じ集合を探す様子を確認してみよう！

“今見ている場所から行けるところまで行って、無理になったら戻る”
イメージ

“今見ている(マス)から(右,下,左,上の順に)行けるところまで行って、無理になったら戻り、次の向きを見る”



実行例

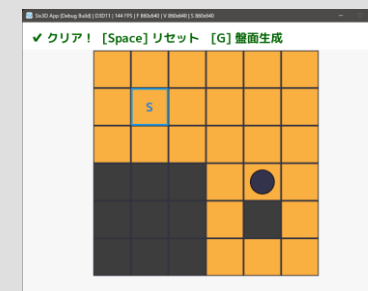
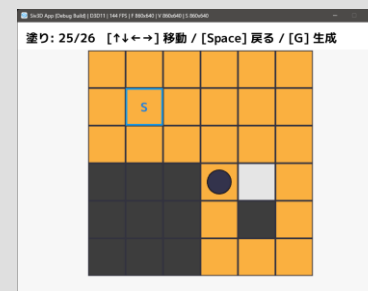
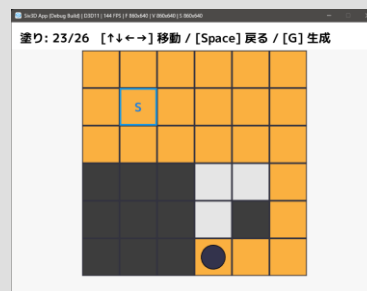
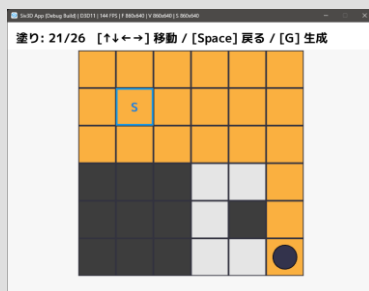
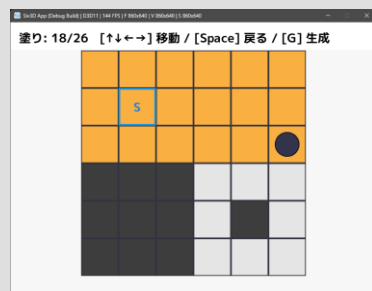
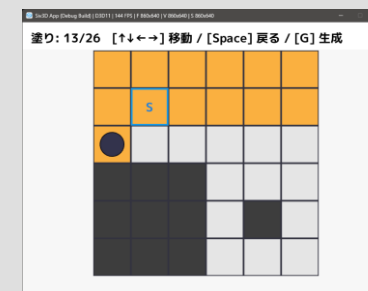
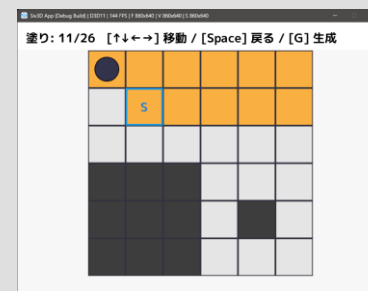
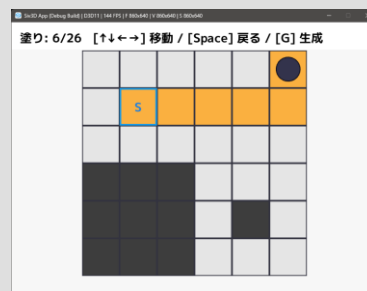
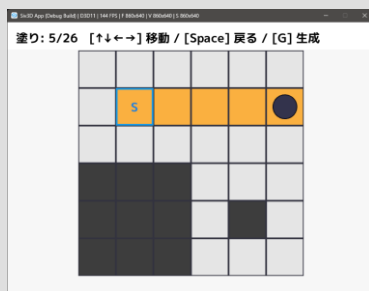
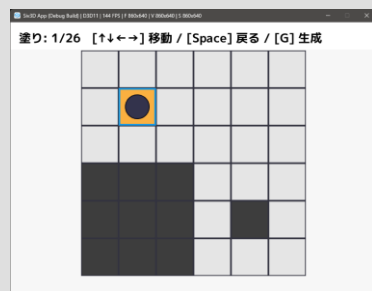


3.実際にパズルゲームの作り方を考えてみる

パズルゲームを新たに思いついたと想定し、それをいかにして実装するか、アルゴリズムによる創意工夫はどこに表れるのかを体験してみましょう

3-1. ゲームのルール

- 滑る駒を動かして一筆書きをするゲーム
- 上下左右キーで移動
- (できれば)盤面をランダムに生成したい



3-2. ゲームの実装

実装するべき要素は 2 種類

- ゲームのロジック(今回は省略)
- 盤面の作り方

3-3. 盤面の作り方

マインスイーパーと違って、こちら側に創意工夫のパートがある

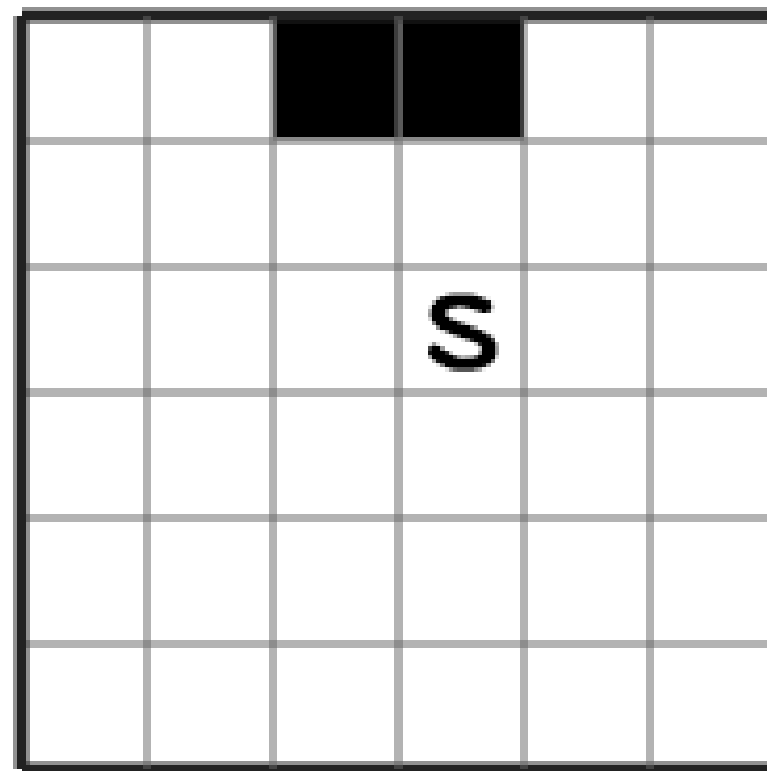
- スタートから作るのは難しい
- ゴールから作るのは意外と簡単

ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

閑話休題

Q. どうやったら解ける？



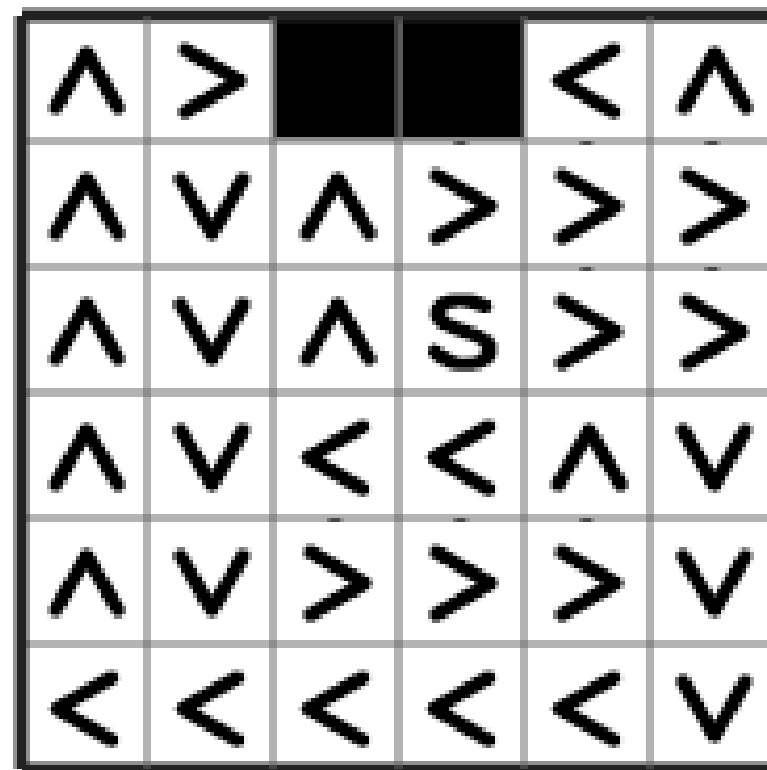
ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

閑話休題

Q. どうやったら解ける？

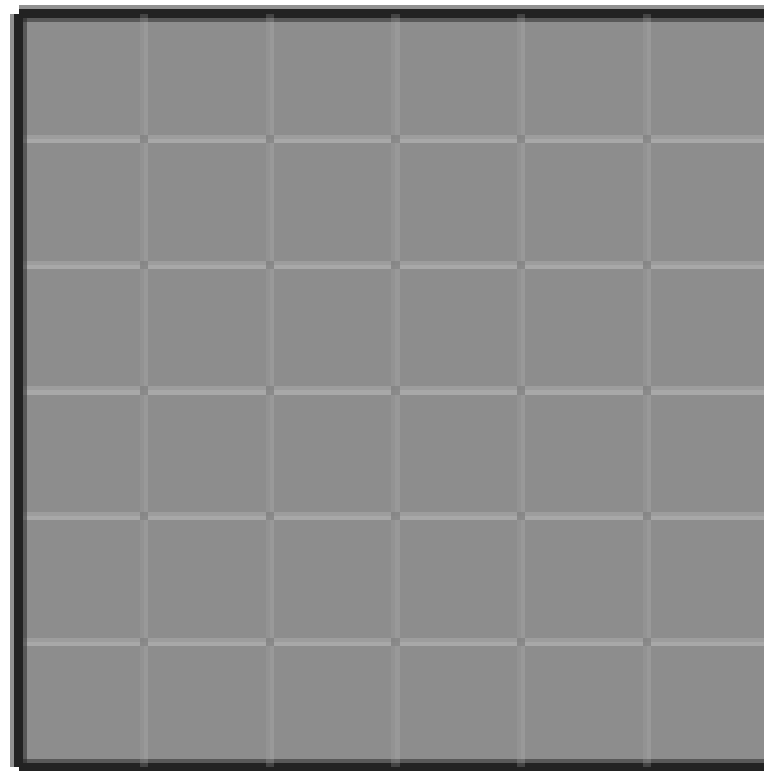
A. こんな感じ



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

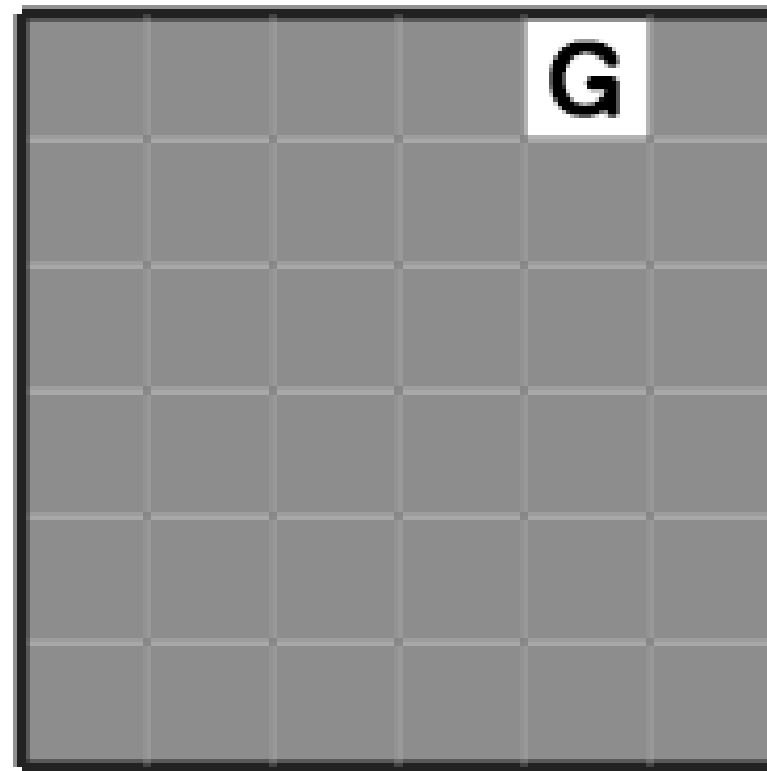
始めに、すべてのマス“壁”か“道”
と仮定する



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

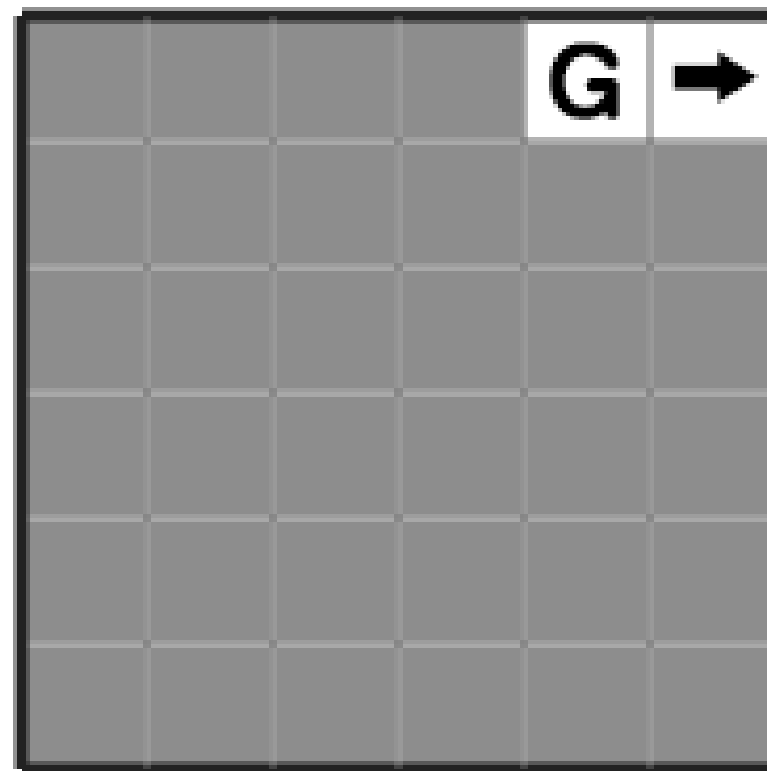
ゴールを一つ決める



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

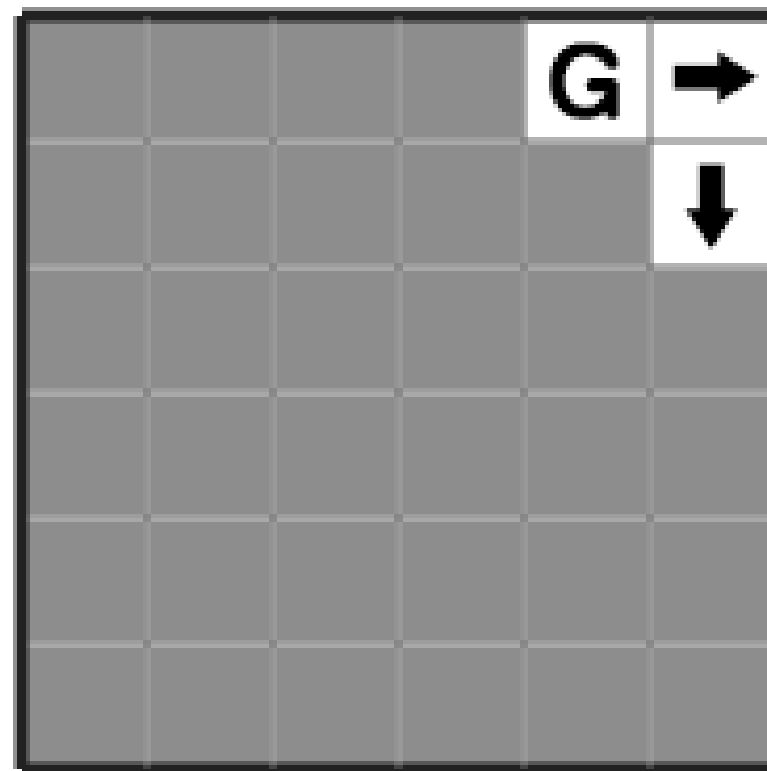
ランダムに移動方向とマス数を決める
右に1マス



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

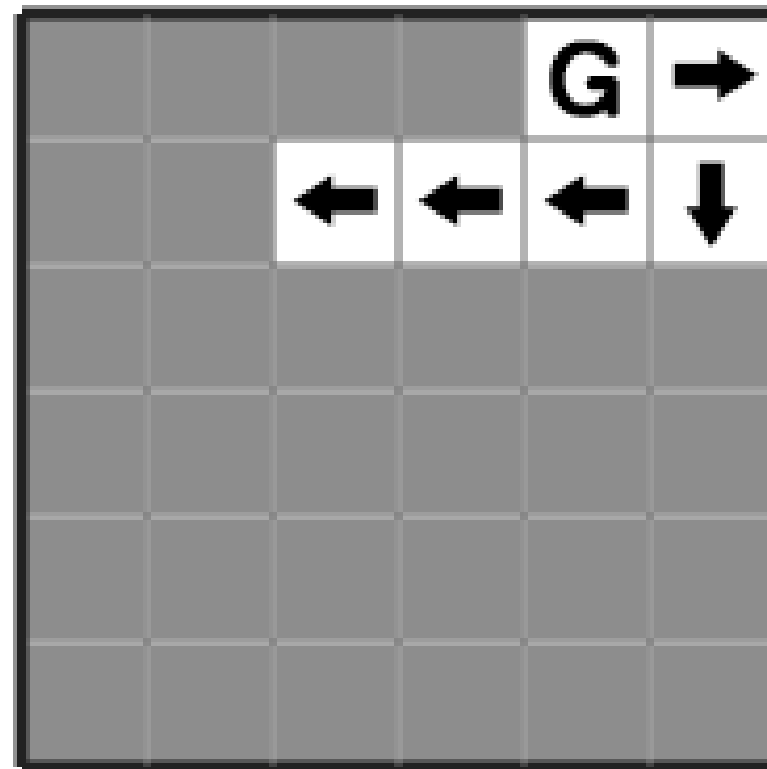
ランダムに移動方向とマス数を決める
下に1マス



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

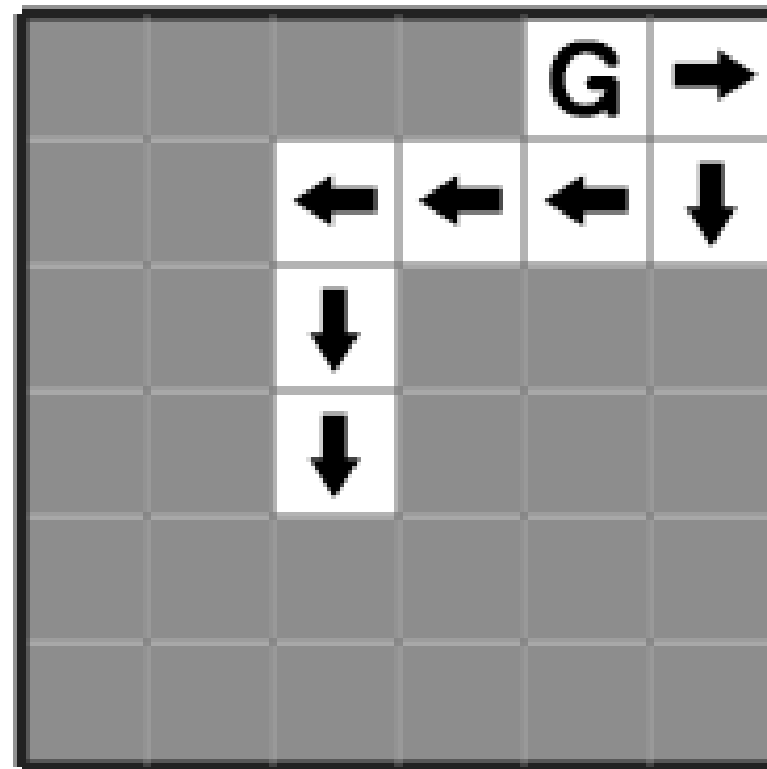
ランダムに移動方向とマス数を決める
左に3マス



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

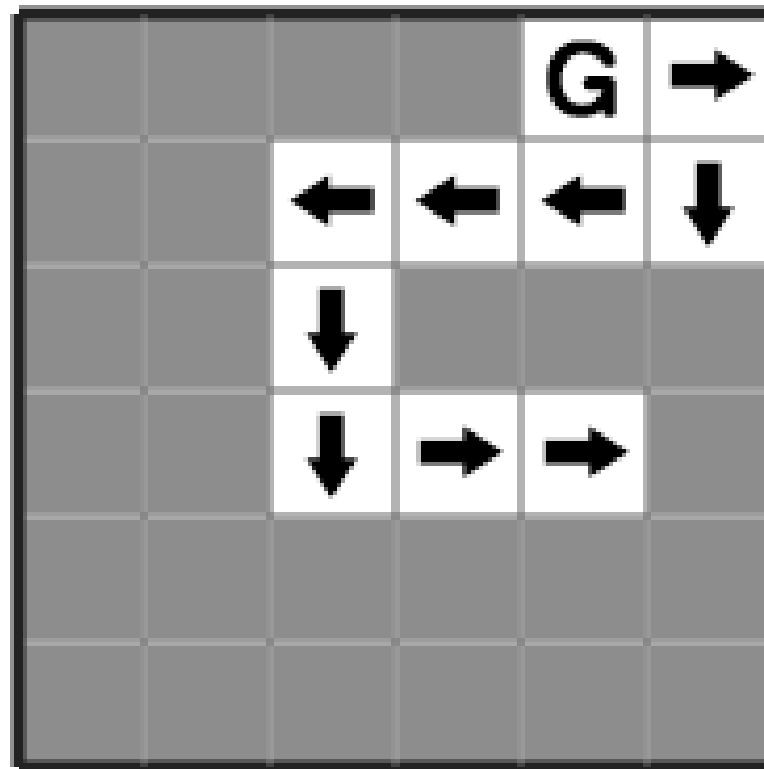
ランダムに移動方向とマス数を決める
下に2マス



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

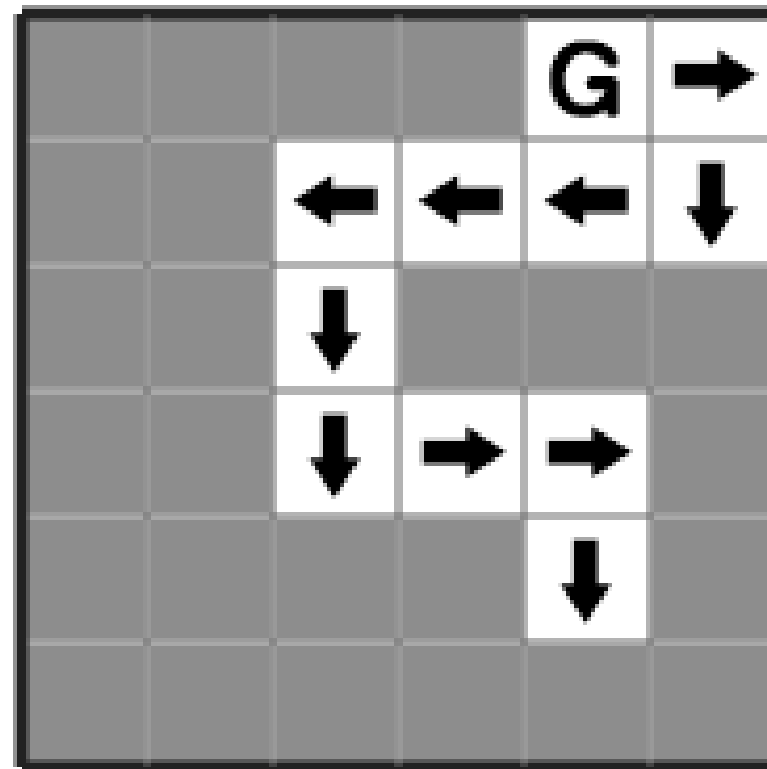
ランダムに移動方向とマス数を決める
右に2マス



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

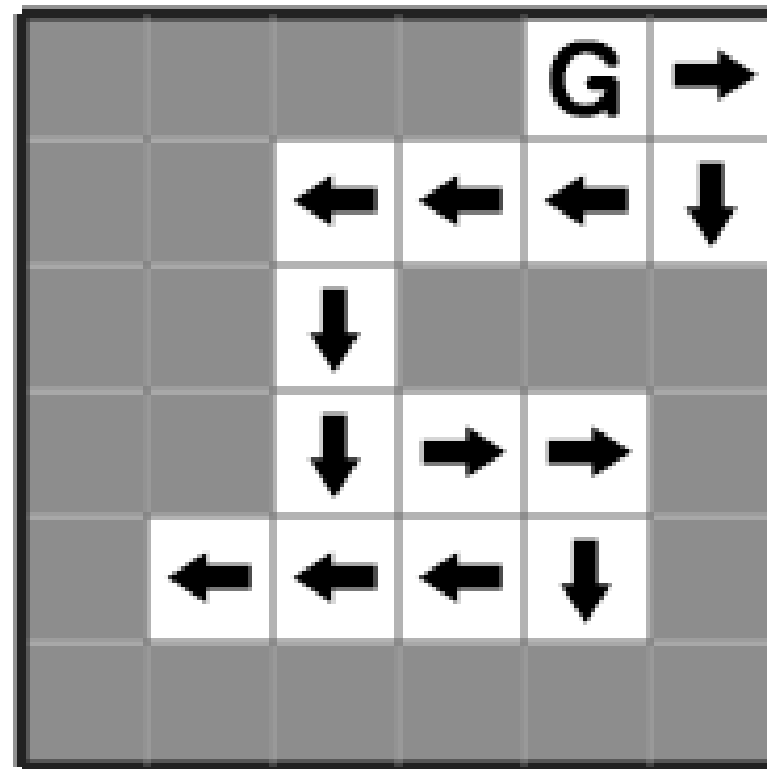
ランダムに移動方向とマス数を決める
(省略)



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

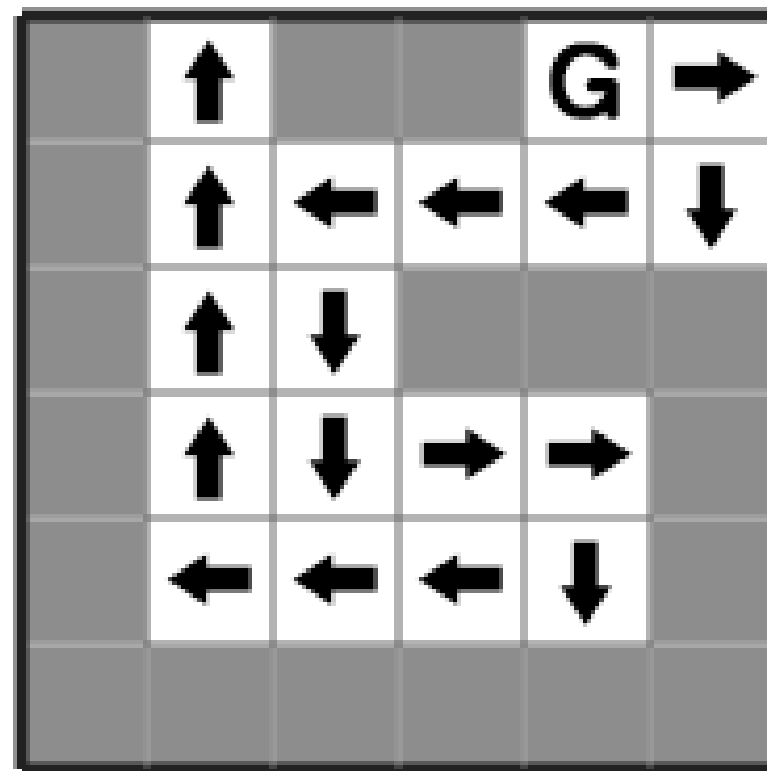
ランダムに移動方向とマス数を決める
(省略)



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

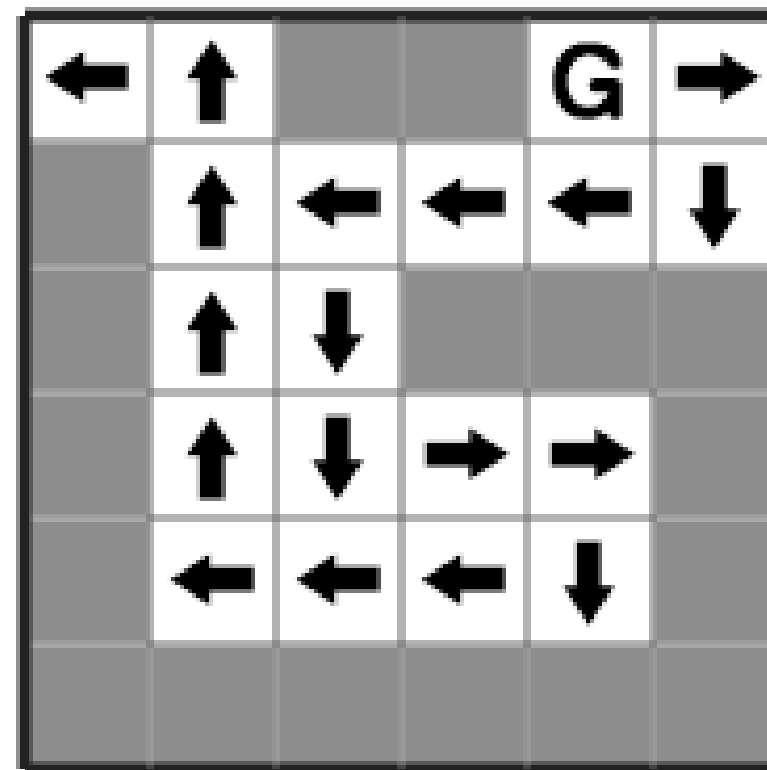
ランダムに移動方向とマス数を決める
(省略)



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

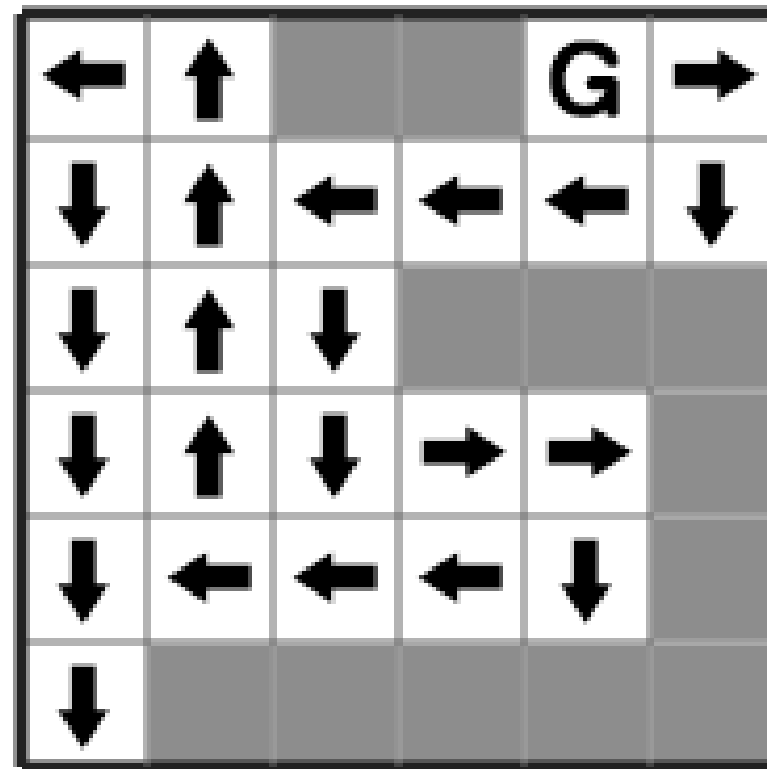
ランダムに移動方向とマス数を決める
(省略)



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

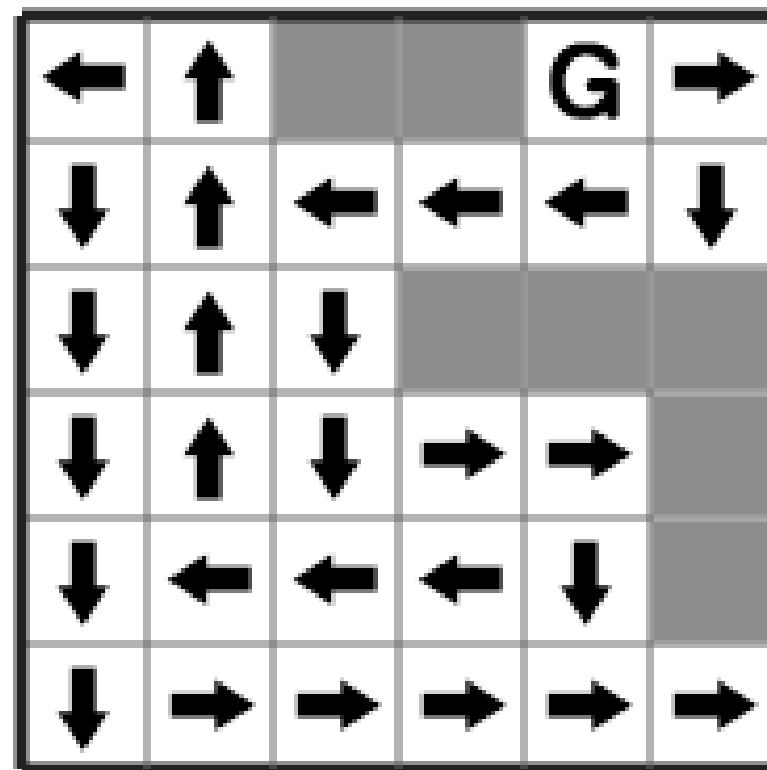
ランダムに移動方向とマス数を決める
(省略)



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

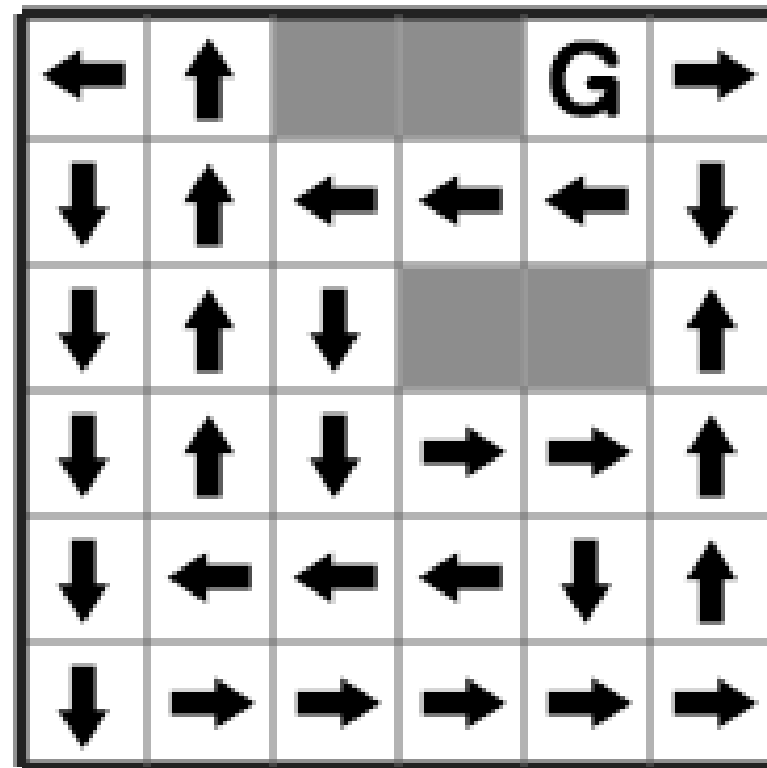
ランダムに移動方向とマス数を決める
(省略)



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

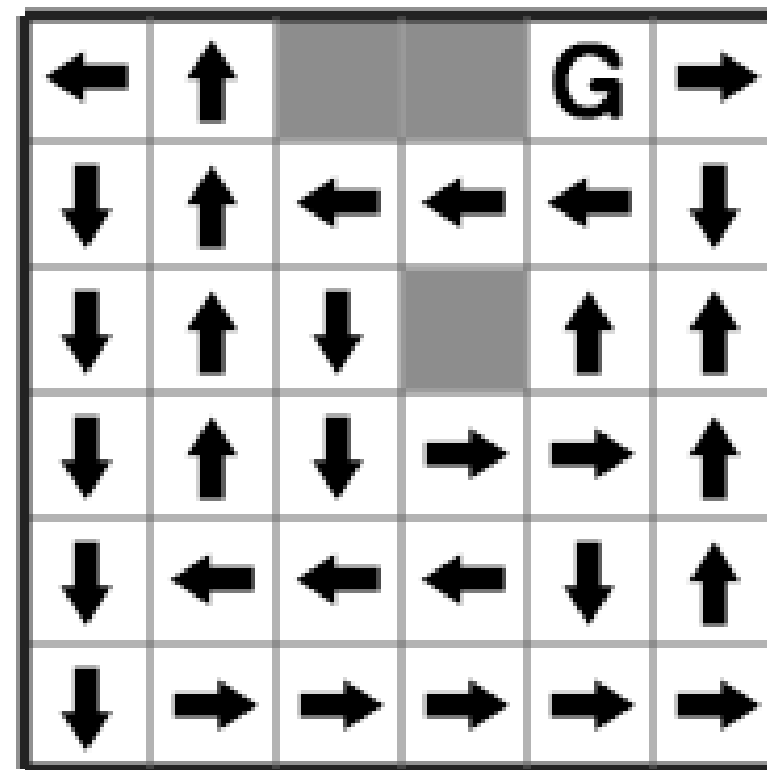
ランダムに移動方向とマス数を決める
(省略)



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

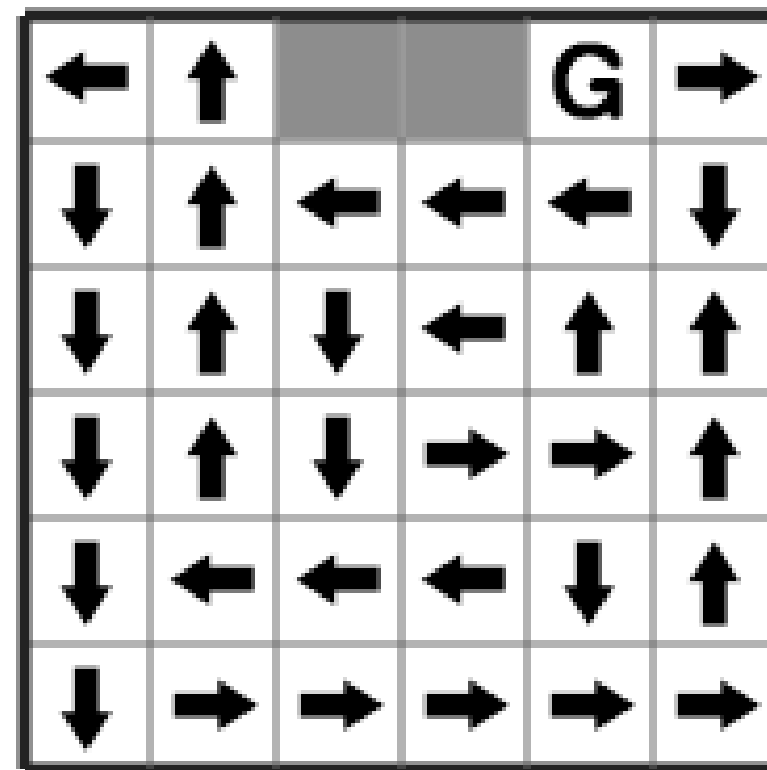
ランダムに移動方向とマス数を決める
(省略)



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

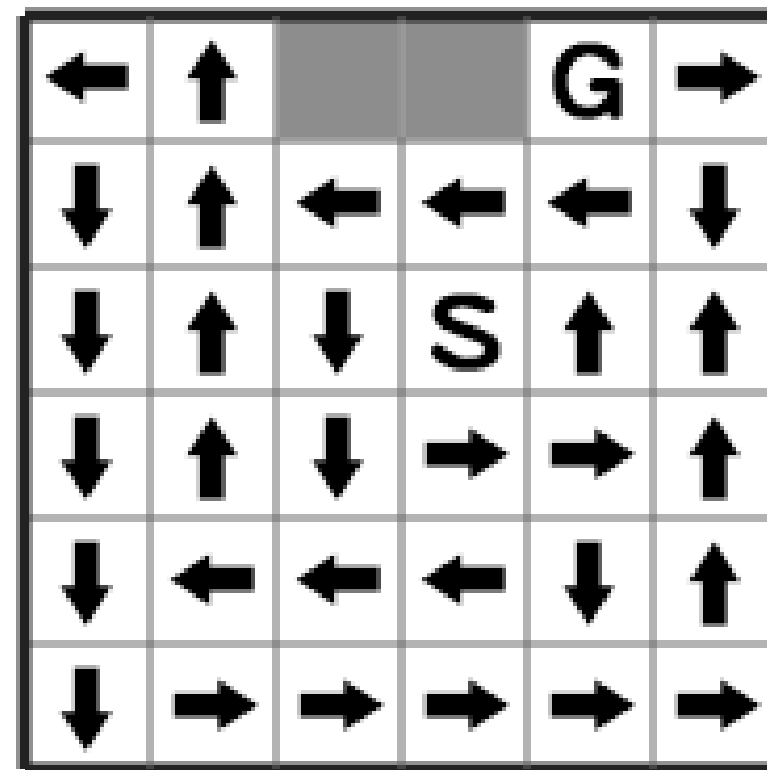
ランダムに移動方向とマス数を決める
(省略)



ゴールから作るのは意外と簡単

例えばこのような盤面を考えてみる

ランダムに移動方向とマス数を決める
詰まったので最後のマスをスタートに
して完成



ゴールから作るのは意外と簡単

```
79 // ランダム生成
80 bool generateOnce(Array<String>& rows, double minCoverRatio) const {
81     Array<String> g(H, String(W, U'#'));
82     Array<Point> carved;
83
84     // ゴールを決める
85     Point goal{ Random(0, W - 1), Random(0, H - 1) };
86     g[goal.y][goal.x] = U'.';
87     carved << goal;
88
89     // 方向ランダム、長さもランダム
90     int stuck = 0;
91     while (stuck < 12) {
92         // 4方向ベクトル (右、下、左、上)
93         Array<Point> Dirs = { Point{1,0}, Point{0,1}, Point{-1,0}, Point{0,-1} };
94
95         Dirs.shuffle();
96         bool moved = false;
97
98         for (const auto& d : Dirs) {
99             Point head = carved.back();
100             // その方向に連続する壁数
101             int maxLen = 0;
102             Point cur = head + d;
103             while (inBounds(cur) && g[cur.y][cur.x] == U'#') {
104                 ++maxLen;
105                 cur += d;
106             }
107             if (maxLen == 0) continue;
108
109             // [1..maxLen] からランダム
110             const int L = Random(1, maxLen);
111             Point p = head;
112             for (int k = 0; k < L; ++k) {
113                 p += d;
114                 g[p.y][p.x] = U'.';
115                 carved << p;
```

```
116             }
117             moved = true;
118             break;
119         }
120         if (!moved) ++stuck;
121     }
122
123     // 壁でない割合の計算
124     const int freeCount = countNonWall(g);
125     if (freeCount < static_cast<int>(minCoverRatio * (W * H))) {
126         return false;
127     }
128
129     // スタートは最後の位置
130     const Point start = carved.back();
131     g[start.y][start.x] = U'S';
132
133     rows = g;
134     return true;
135 }
```

工夫を凝らせば一見難しかったものも実現できる

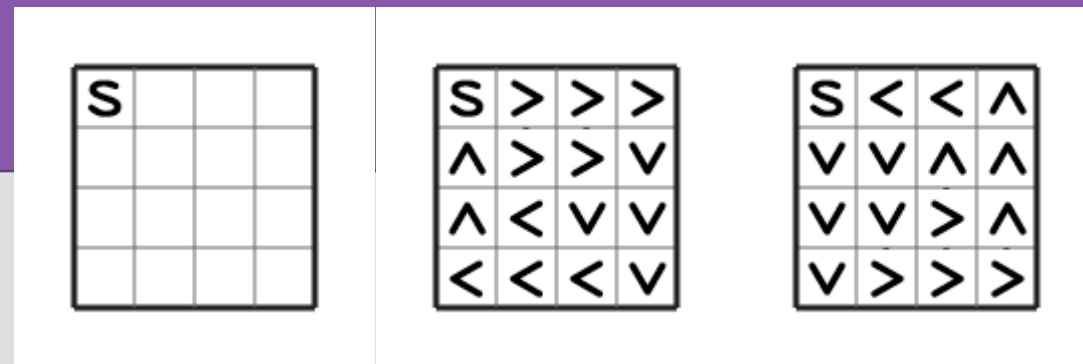
- この考え方は競技プログラミングに限らず開発などの場面でも絶対に登場する
- （そういう意味でも開発沼の人々に競プロをオススメしにくい...競プロも面白いんだけどね）

後はお好みで

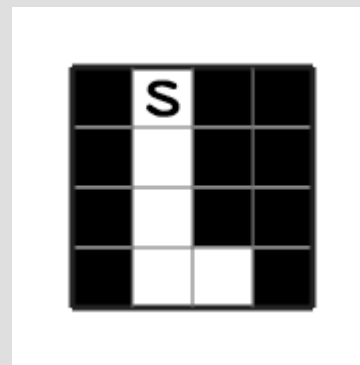
より洗練するために

1. 解の一意性判定
2. “盤面の美しさ”の定義
3. より良い盤面生成アルゴリズム

などを実装すると良い



1. 複数パターンある盤面の例
→想定より簡単なやり方があるかもしれない



2. 盤面が美しくない例
→壁が多すぎてつまらないかもしれない
→答えが目に見えてるかもしれない

解の一意性判定

解く方は素直にできる

1. スタートから移動する方向を決める
2. 壁に当たるまで滑る
3. 2通りの移動方法のうちどちらかを試す

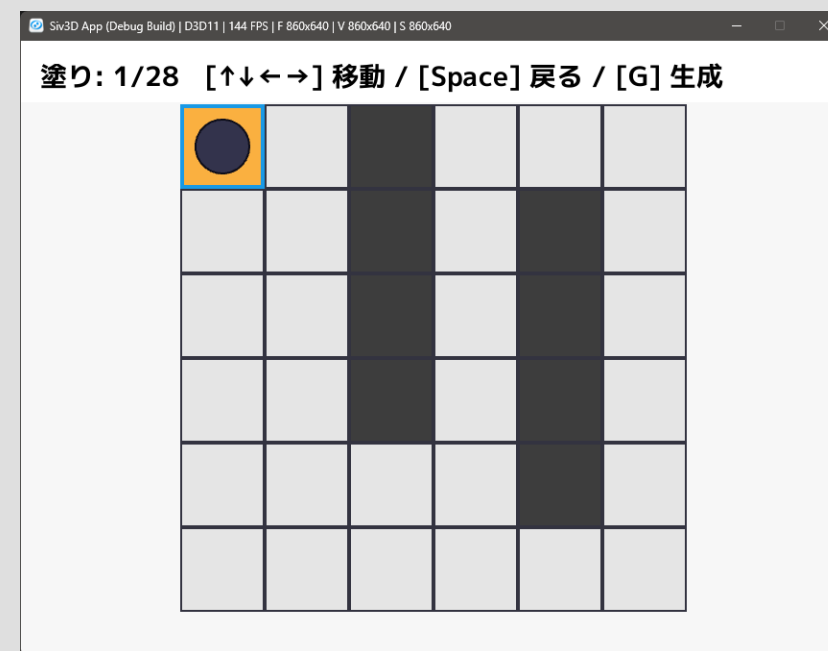
以上を全探索して、1通りしかないかを判定する

```
std::function<void(Point, int)> dfs = [&](Point head, int painted) {  
    // 解が2つ見つかったら終了  
    if (solutions >= 2) return;  
    if (painted == nonWall) {  
        ++solutions;  
        return;  
    }  
    // 可能手を列挙  
    Array<Array<Point>> moves;  
    for (auto d : Dirs) {  
        Array<Point> path;  
        Point cur = head + d;  
        if (isObs(cur)) continue;  
        while (!isObs(cur)) { path << cur; cur += d; }  
        if (!path.isEmpty()) moves << path;  
    }  
  
    for (const auto& path : moves) {  
        for (const auto& p : path) vis[p.y][p.x] = true;  
        dfs(path.back(), painted + static_cast<int>(path.size()));  
        for (const auto& p : path) vis[p.y][p.x] = false;  
        if (solutions >= 2) return;  
    }  
};  
  
dfs(start, 1);  
// 一意なら true, そうでなければ false (元々 1 通りは存在する)  
return (solutions == 1);
```

“盤面の美しさ”

配布しているプログラムでは実装していないが、このアルゴリズムによって生成される盤面は面白みに欠けるものも多い

- 壁マス数を減らしたり、曲がる回数が多いものだけを採用したりするとより美しくなるかも？



あまりにも素直に解けてしまう例

より良い盤面生成アルゴリズム

- 盤面を大きくすると、実はこのアルゴリズムは生成に時間がかかる (or 生成できなくなってしまう)
 - 探索アルゴリズムは計算量を見積るのが難しいので、すぐかけるようになると実験がしやすい (可能/不可能の判断が実装からわかるように)
 - 開発のマインドとしても、ある程度の範囲で妥協するのも大事かも

逆に計算量の見積もりがうまくなると、妥協点の判断もしやすくなるので、知っておくとお得！ (例：これ以上改善してもほぼ時間が変わらないから、このロジックは適当でいい, 等)

※様々な都合を加味すると、実行してみないと実際どうなのかわからないことも正直多いです。

4.オマケ：ひょっとしたら使えるかもしれない問題の話

せっかくですので、ひとつアルゴリズム関連の話も出来たらといったところです。ここでは、“SAT” と呼ばれる問題について解説できます。

SAT :充足可能性判定(SATisfiability problem)

「命題論理式が与えられた場合、それに含まれる変数の値を偽か真にうまく定めることによって全体の値を'真'にできるか」という問題(wikipediaより)

命題論理式の例 $(x_1 \vee x_2) \wedge (x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$

「 x_1 が真か x_2 が真」かつ「 x_1 が真か x_2 が偽」かつ「 x_1 が偽か x_2 が偽」→「 x_1 が真で x_2 が偽」の時に全体は'真'となる

SATとペンシルパズルは相性がいい

ペンシルパズルを解くときには、実は似たようなことを頭で考えていると思いませんか？

例えば数読であれば

「マスAに1が入る」, 「マスAに2が入る」, ..., 「マスAに9が入る」, ...といった729(81x9)個の変数があり、

行Xには「1~9までの数が1つずつ入る」

「マス α には1が入る」かつ「マス β には1が入る」...「マス η には1が入る」かつ

「『マス α には1が入る』または『マス β には1が入る』」かつ... と記述できる(?)

X	5	3	A		7				
	6			1	9	5			
		9	8					6	
	8				6				3
	4			8		3			1
	7				2				6
		6					2	8	
				4	1	9			5
	α	β	γ	δ	8	ε	η	7	9

数読はSATに置き換えることができる

2-SATは線形時間で解くことができる

$$(x_1 \vee x_2) \wedge (x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$$

- 例で挙げたような2-SAT (「または」で繋がっている変数が2つ)は $O(n)$ で解くことができます
 - 競プロでたまに登場するアルゴリズムですが、言いたいことと離れているので割愛

一般にSATはNP完全問題

「NP完全」≡多項式時間で解が求められるアルゴリズムは知られていない

検証は簡単（真か偽かを代入するだけ）だが、解を決定的に求めることが難しい

SATを頑張って解いてくれるアルゴリズムはある

参考になりそうな文献「制約ソルバーとペンシルパズル」
(<https://zenn.dev/semiexp/books/e559bbd5ab00a2>)

一度自分の作ったパズルゲームが適用できないか/適用できた場合はツールの判定を活用して生成できないか、一考の価値は（ほんの少しだけ）あるような気がします。

質疑応答 & おわりに

ご清聴ありがとうございました

今後のためにアンケートに回答していただけるとありがたいです